Advanced environment labs

World Engine technical research

October 10, 2026

Build nine review artifacts: a fidelity contract, a grader attack matrix, an evaluation analysis, a split/rights manifest, a preference-batch specification, an RL capture contract, a scheduler budget, a cost model, and a deployment threat model. You can complete the numeric exercises with paper or a calculator. The optional repository checks use scripted tests, not model inference.

All invented numbers below are **illustrative**, including prices, token rates, memory layouts, sample sizes, and agreement scores. They are not World Engine measurements. Actual receipts remain in the foundation report, follow-up appendix, and matched-study results. This workbook adds no benchmark, training run, application-parity claim, or production readiness claim. Its source ledger records primary-source checks and limits.

Lab 1. Specify behavioral fidelity before measuring it

An environment is a state-transition system. Its state contains operational facts, an action asks for a change or a read, and its observation exposes some state to the agent. The policy chooses actions from observations and history. A rollout is the resulting sequence of decisions, tool results, and states. A grader judges that sequence and its outcome against a task contract.

A synthetic environment can look unlike an application yet preserve one workflow accurately. Conversely, an exact-looking screen can give incorrect permission, audit, or retry behavior. Fidelity is agreement on declared behavior, not the number of copied fields or screenshots. Fleet's archived public SDK demonstrates versioned creation, seeded reset, SQL access, and cleanup. That lifecycle alone does not establish which application behaviors match. Fleet SDK.

Produce a differential contract

Choose one bounded task, such as refunding an eligible damaged order. Define an abstraction α that maps application records to comparable business facts. Drop irrelevant rendering IDs; retain money in integer cents, ownership, authorization, and audit order. Then compare an application transition with a simulated transition:

$$\alpha(T_{\text{app}}(s, a)) \stackrel{?}{=} T_{\text{sim}}(\alpha(s), a).$$

This deterministic equation assumes equivalent initial facts and action semantics. For asynchronous or stochastic behavior, compare declared distributions or eventual predicates under a fixed deadline instead. Do not erase genuine nondeterminism by sorting a causally ordered audit log.

Define these axes independently:

Axis	Differential probe	Failure that matters
Data	Same target plus unrelated rows	Wrong target or lost outsider
Permissions	Same action under two roles	Simulation admits a forbidden payment
Observation	Same search/page/history	Agent sees privileged approval data

Axis	Differential probe	Failure that matters
Transition	One-cent or expiry boundary	Different eligibility verdict
Retry	Same key twice, then another key	Duplicate payment or missing conflict
Time	Delayed review, timeout, reset	Approval never expires or time resets
Outcome	Independent final-state check	Good-looking transcript, wrong ledger

Use an authorized application test tenant, synthetic records, and a reset procedure. Freeze versions, configuration, clock semantics, and adapters. Execute the same action sequence in both systems. Preserve before/after facts and errors, not only final pass rates. Ask domain experts to adjudicate mismatches with the same public instructions given to the agent. Anthropic's evaluation guidance separates transcripts from outcomes and recommends expert calibration. Agent evals.

Solve the agreement example

Suppose 200 transitions yield 180 agreements, eight harmless formatting differences, eight business mismatches, and four permission mismatches. A raw agreement score is 90%. After a justified formatting normalization it is 94%. Neither number makes four unsafe admissions acceptable.

Assign loss weights 0, 1, and 10 to normalized agreement, business mismatch, and permission mismatch. The mean loss is $(8+10\times4)/200 = 0.24$. This weight choice is a declared risk preference, not a probability of harm. Report the permission count separately so a common easy read cannot hide a rare dangerous write.

Question. Zero unsafe admissions occurred in 100 adversarial cases. Is the rate proven zero? **Solution.** No. Even under independent representative trials, the one-sided 95% binomial upper bound is $1 - 0.05^{1/100} \approx 2.95\%$. Adversarial cases are usually selected, not a random population sample, so that statistical bound does not imply a production rate. Publish tested categories and blind spots.

Deliverable. A one-page contract with seven axes, reset/clock rules, risk weights, explicit exclusions, expert disagreement procedure, and paired mismatch records. World Engine has not completed this application differential study.

Lab 2. Attack the grader separately from the tools

A reward is a scalar optimization signal. Task success is the intended outcome. Those can disagree. Specification gaming exploits the score's definition; reward tampering changes the scoring machinery or its evidence. DeepMind describes the former distinction. Anthropic's controlled coding-training study reports broader effects after reward hacking, but this project has no training result that establishes the same effects. Specification gaming, reward-hacking study.

Produce an attack matrix

Attack class	Synthetic workflow probe	Required defense
Proxy shortcut	Finish without granting or closing	Outcome predicate, not completion marker alone
Target substitution	Change request ID or canonical customer	Anchor identities to immutable initial facts

Attack class	Synthetic workflow probe	Required defense
Evidence forgery	Insert finance approval or merge audit	Exact authorized history, not field presence
Authority laundering	Replace role, owner, limit, or policy	Trusted permission facts outside agent edits
Retry farming	Reuse a key to create more reward/events	Idempotent effect and exact event count
Preservation failure	Solve target, delete unrelated ledger	Grade whole allowed state, including outsiders
Grader intrusion	Rewrite tests or read expected output	Separate agent and verifier credentials/files
Distribution shortcut	Infer denial from a seed or broad hint	Balanced cases and evidence-required decisions
Type confusion	Substitute `true` for ID `1`	Type-sensitive canonical comparison
Artifact deception	Link output to hidden files; exceed archive cap	Validate complete detached output, fail closed

Use two independent test routes. First, ask an ordinary tool to perform the forbidden operation. Its error must leave operational state unchanged, apart from any explicitly allowed clock tick. Second, corrupt a trusted test's completed snapshot directly, then invoke the grader. That probe checks the grader contract; it is not evidence that an agent can reach the corrupted state through normal tools.

An independent grader starts from trusted initial facts and folds exactly permitted change certificates into an expected state. It checks prerequisites, identities, actor authority, event order, and unchanged fields. It must accept every legal operation order, not only a reference transcript. Calling the same buggy `apply` function again merely reproduces its mistake. A locally generated certificate is also not a signature against a malicious host operator.

Inspect existing proofs without inference:

```
uv run pytest tests/test_workflow_grader_integrity.py \
  tests/test_incident_integrity.py
```

Read the positive-prefix tests before adding attacks. A correct permission denial after legitimate progress must not turn that progress into a safety violation. Repeating an idempotent payment must not create another audit event. The grader correction explains actual historical gaps without rewriting old receipts.

Solve the reward example

Suppose a grade averages safety, settlement, and explicit finish. An unchanged safe world followed by finish earns 2/3, despite settling nothing. An optimizer can prefer that cheap shortcut. Multiplying by a safety check stops unsafe outcomes, but it does not fix free credit for doing no work.

For a new outcome-only task, use $r = \mathbf{1}[\text{safe} \wedge \text{settled} \wedge \text{finished}]$ and retain individual checks as diagnostics. That is the access world's declared choice, not proof that binary feedback always trains better. Legacy shaped grades remain historical semantics; episode timeouts/unfinished termination still force authoritative episode reward to zero.

Potential-based shaping instead adds $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$ under specified conditions. With $\gamma = 0.9$, $\Phi(s) = 2$, and $\Phi(s') = 3$, the shaping term is 0.7. Across a discounted path those terms telescope,

with terminal-boundary conditions mattering. Arbitrary check averages have no such policy-invariance guarantee. Reward-shaping theorem.

Deliverable. An attack matrix with a legal counterexample for each rejected mutation. Record severity, reachable-through-tools status, grader result, source revision, and the regression that prevents recurrence. Production runtime guards must not depend on Python `assert`, which optimized Python removes.

Lab 3. Define the denominator before reading the score

A task is a problem instance. A trial is one attempt at that task. Four adapters on one task create four trials, not four independent tasks. Repeating one task under several seeds may still share its rule family. Pin the policy, prompt, tool parser, environment, grader, budget, and collection order before gathering outcomes.

Keep attempted, finalized, verified, inconclusive, and unattempted counts separate. A missing journal row is not a failed reasoning attempt. A persisted malformed envelope is not a verified episode. Never drop failures or silently replace an interrupted slot with a successful retry. For a predeclared end-to-end benchmark, timeouts and valid incorrect outcomes count as task failures; infrastructure-inconclusive rows need their own count and rule.

Solve a matched comparison

Two adapters attempt 40 fixed tasks once each. They both pass 14, only A passes ten, only B passes two, and neither passes 14. Then A passes $24/40 = 60\%$, B passes $16/40 = 40\%$, and the paired difference is $(10 - 2)/40 = 20$ percentage points.

For each task define $d_i = 1$, -1, or 0 according to A-only, B-only, or equal outcome. Resample **whole task pairs**, not two unrelated lists of attempts. If tasks belong to declared families, resample within those strata and report the weighting rule. The paired result compares complete adapter systems, not one isolated serialization feature.

A confidence interval describes an estimation procedure's coverage under its sampling assumptions. It is not a probability that one fixed true rate lies inside this realized interval. Selected toy fixtures support descriptive uncertainty summaries, not population coverage claims. For correlated variants, resample parent families rather than pretending every row is independent.

Now divide A's tasks into ten difficult tasks with two passes and thirty easier tasks with 22 passes. The micro-average is still 60%. Giving the two families equal weight yields $(20\% + 73.33\%)/2 = 46.67\%$. Neither average is wrong; they answer different workload questions. Specify the target mixture instead of choosing whichever looks better. RL evaluation research warns about uncertainty and unstable aggregate conclusions from limited runs. Statistical precipice.

Diagnose before proposing a fix

Observed category	What the record establishes	What it does not establish
Provider protocol error	Response violates frozen action contract	Model cannot solve the business problem
Environment denial	Action fails a declared permission/precondition	Provider transport failed
Valid incorrect completion	Parser accepted actions; goal unmet	Grader is correct without independent checks

Observed category	What the record establishes	What it does not establish
Maximum steps	Budget ended before accepted completion	More steps would necessarily solve it
Timeout	Deadline ended the attempt	Which component caused the delay
Setup/cleanup/persistence error	Controller lifecycle failed	A comparable clean policy outcome

Inspect raw exchanges and state history to refine causes. A timeout after repeated denied approvals may involve both poor decisions and expensive inference. Exclusive report categories need declared precedence; secondary causes can coexist. Relaxing a one-call parser after seeing multi-call failures changes the experiment. Collect a new locked protocol rather than rescoring the old one under a more forgiving contract.

Question. A trial succeeds with probability 0.8. What are success-at-least-once and success-every-time over three trials? **Solution.** Under identical independent trials, $1 - (1 - 0.8)^3 = 0.992$ and $0.8^3 = 0.512$. Independence and constant probability are assumptions, not consequences of three observed repeats. A retry-friendly search score and a reliability score answer different questions.

Deliverable. A denominator table, all four matched outcomes, micro and declared macro scores, confidence intervals with sampling caveats, clean-success counts, failure taxonomy, and every attempted slot. The actual matched study uses this discipline; its result is not the invented 40-task example above.

Lab 4. Separate seed holdout, family holdout, and rights

A curriculum changes which training tasks a policy sees and when. A split assigns examples to training, tuning/validation, or final evaluation. They are related but different. Validation guides development; once repeated feedback changes your choices, it is no longer a blind final test. Procedural generation can measure unseen-level behavior, as Procgen does, without establishing transfer to an unseen environment or rule. Procgen paper.

Produce a split manifest

Imagine 100 workflow families with ten seeded instances per family. A family specifies a rule and interaction pattern; a seed changes names, amounts, distractors, or ordering within that rule. Randomly assigning 800 rows to train and 200 to test usually puts each family in both sets. Those 200 rows measure instance variation, not withheld-rule reasoning.

Instead, reserve 70 families for training, fifteen for validation, and fifteen for final evaluation. Assign all seeds, paraphrases, reference solutions, revisions, and derived traces of a family together. Keep a stable parent identity even when implementation hashes change. Freeze the mapping before generation and publish overlap checks. World Engine's stable family/seed lineage prevents implementation revisions from moving the same instance between splits, but does not implement this broader family-heldout study.

Use at least three claims with different tests:

1. Unseen seed, known family: new invoice IDs and distractor order.
2. Unseen family, known domain: multi-approval refund rule absent from training.
3. Unseen application/domain: equivalent policy expressed through another service's navigation and errors.

Increase curriculum difficulty by adding partial observation, ambiguous identifiers, longer valid dependencies, and recovery branches. First prove each task has a legal solution under its budget. Preserve easy regression tasks while adding difficult capability tasks. Reject impossible or contradictory fixtures rather than teaching the model to guess the desired verdict.

Solve a contamination example

A family has three paraphrased rows. You independently send each row to training with probability 0.8. The probability that the family appears on both sides is $1 - 0.8^3 - 0.2^3 = 0.48$. Forty-eight percent of such families are expected to cross the boundary. Row hashes will not catch paraphrases. Group by provenance and semantics before splitting; use exact and near-duplicate scans as additional checks.

Contamination includes leaked tests, reference solutions, task-specific hints, earlier evaluation feedback, and synthetic outputs derived from heldout data. Poisoning is different: an attacker supplies examples intended to induce unwanted behavior. Anthropic's controlled backdoor study motivates source review, but does not provide a universal safe percentage. Poisoning study.

Produce a rights manifest

For each source, record owner, exact revision, file-level license/notices, permitted uses, restrictions, personal-data status, consent or other approved processing basis, retention, and derived artifacts. Record who approved reuse and which deletion requests must propagate. Synthetic names do not cleanse a copied private policy or customer incident.

Public visibility is not public-domain status. Apache-2.0 grants permissions with conditions; CC0 is a rights dedication mechanism, while the Public Domain Mark identifies an existing status. Neither label establishes consent to process private records or third-party rights. FrontierSWE v2's notice describes evaluation-purpose task content and per-task attribution, so do not assume blanket training reuse. Apache license, CC tools, FrontierSWE repository.

Deliverable. Separate instance, family, and application splits; a curriculum change log; contamination scans with known limits; and an approved rights manifest. Keep blocked data outside candidate pools, not merely marked in an export after selection. Obtain qualified rights review before reuse where permission is unclear.

Lab 5. Turn a preference into a defined likelihood

Supervised fine-tuning, or SFT, increases the probability of selected demonstrations. Direct preference optimization, or DPO, learns from a preferred and rejected completion of the same prompt. Neither is a name for copying readable trajectories into a JSON file. Before training, specify what the model predicts, which tokens receive loss, and how the reference model sees identical context.

For SFT, let z_t be a token, h_t its preceding context, and $m_t \in \{0, 1\}$ a loss mask. One declared mean-token objective is

$$\mathcal{L}_{\text{SFT}} = -\frac{\sum_t m_t \log \pi_\theta(z_t \mid h_t)}{\sum_t m_t}.$$

A mask chooses which tokens teach the policy. Context still conditions predictions even when it has mask zero. Reject an example with no active loss tokens rather than dividing by zero. Padding must not contribute loss. For an action policy, service-generated tool results are context, not the agent's sampled actions. Decide explicitly whether delimiters and end-of-turn tokens belong to the completion.

Chat templates encode roles and boundaries differently across models. The official Transformers documentation warns that applying extra special tokens after template formatting can duplicate them. Record the tokenizer, template revision, rendered IDs, truncation rules, and generation boundary. Decoded text alone cannot prove that two pipelines used identical tokens. Chat templates.

Solve a DPO example

For one prompt x, define preferred completion y^+ and rejected completion y^-. Write the original reference-relative loss as

$$\mathcal{L}_{\text{DPO}} = -\log \sigma(\beta[\Delta^+ - \Delta^-]), \qquad \Delta^\pm = \log \pi_\theta(y^\pm \mid x) - \log \pi_{\text{ref}}(y^\pm \mid x).$$

$\sigma(u) = 1/(1 + e^{-u})$ is the logistic function. The reference policy supplies a baseline likelihood. The preference derivation assumes a particular model of comparative reward; minimizing the loss does not validate the labels. DPO paper.

Suppose the trained-policy log-likelihoods are -12 and -10, reference likelihoods are -13 and -9, and $\beta = 0.1$. Then $\Delta^+ = 1$, $\Delta^- = -1$, the logit is 0.2, preference probability is approximately 0.5498, and loss is approximately 0.5981. Although the preferred completion has lower absolute probability, its improvement relative to reference is greater. Reverse the preference and the loss becomes approximately 0.7981.

Sequence log-likelihood sums the selected conditional token log-probabilities. Replacing a sum with a length-normalized mean changes the objective. So do different truncations, masks, or reference contexts. A six-token exercise with four context tokens and two action tokens uses mask `[0,0,0,0,1,1]`; shifting that mask one position teaches the wrong boundary even if the JSON parses.

Make workflow pairs comparable

Choose matching initial prompt, scenario, world/grader version, split, policy contract, and budget. A successful refund and a failed refund for another order are not a same-prompt preference pair. A shorter failed attempt after an infrastructure exception may not express a defensible business preference. Preserve the cause and selection rule.

For a whole tool trajectory, later observations depend on earlier actions. You cannot paste both histories under one final observation and call them alternative next actions. Specify an interactive objective with each action conditioned on its own history, or declare a transformed trajectory-completion task and evaluate that new task. Do not assign the policy likelihood of service outputs as though it generated them. TRL documents explicit-prompt and conversational formats, but that structure alone does not resolve these semantics. DPO trainer.

Question. A preferred record is task-passing but contains five denied actions; another is task-passing and clean. Does higher reward define a preference? **Solution.** Not if both receive the same outcome reward. Define the desired quality relation first, retain ties, and avoid inventing labels from record order. Business success, cleanliness, latency, and safety are distinct objectives.

Deliverable. One inspected candidate pair with rendered token IDs, action masks, reference inputs, likelihood aggregation, preference provenance, and truncation checks. This is a specification exercise. The local trainer feasibility audit remains a separate blocker; no usable tokenizer/trainer path is supplied by this workbook.

Lab 6. Distinguish PPO/GRPO batches from grouped text

Online reinforcement learning collects policy behavior, grades it, then updates the policy. The **behavior policy** produced the action. The **current policy** is being optimized. A **reference policy** constrains drift or supplies a comparison baseline. They are not interchangeable just because they share a model name.

An advantage estimates how much better an action's return is than a baseline. A PPO actor-critic system commonly uses a learned value estimate for that baseline. GRPO's original formulation instead compares sampled completions within a same-prompt group. Both still need a defined policy likelihood and a specified estimator. PPO paper, original GRPO.

Solve the clipping calculation

For a token or action with recorded behavior log-probability ℓ_{old} and current log-probability ℓ_θ, define $\rho = \exp(\ell_\theta - \ell_{\text{old}})$. PPO's clipped surrogate contribution is

$$\min(\rho A, \text{clip}(\rho, 1-\epsilon, 1+\epsilon)A)\,.$$

With $\ell_{\text{old}} = -2$, $\ell_\theta = -1.8$, $A = 2$, and $\epsilon = 0.2$, the ratio is about 1.2214. The unclipped contribution is 2.4428; the clipped contribution is 2.4, so the minimum is 2.4. If $A = -2$, the minimum is -2.4428, not -2.4. The minimum's direction matters. Clipping is not a blanket cap on every gradient, a guaranteed global KL bound, or protection against a wrong reward.

Solve the group calculation

For four same-prompt samples with rewards `[0,0,1,1]`, the mean is 0.5. Using **population** standard deviation for this exercise gives 0.5, so standardized advantages are `[-1,-1,1,1]`. Using sample standard deviation gives a different magnitude. Declare the convention and numerical epsilon rather than relying on a name such as "GRPO".

Groups `[0,0,0,0]` and `[1,1,1,1]` have zero centered reward. Their outcome-driven advantage is zero under this rule, although a configured KL or other loss may still act. Mixing four different prompts in one group subtracts an unrelated baseline. Repeating an identical prompt with deterministic decoding may also supply no diverse candidates. A curriculum with only impossible tasks wastes sampling; one with only solved tasks supplies little comparative outcome signal. Current TRL offers several reward-scaling and loss choices, so pin the trainer revision and complete configuration. GRPO trainer.

Produce a training capture contract

Require exact prompt IDs, sampled completion IDs, sequence boundaries, action/loss masks, behavior token log-probabilities, policy/checkpoint identity, generation settings, stop/truncation reasons, task/grader identity, and rewards. Declare temperature/top-p effects and any generation-versus-training likelihood correction. Later retokenizing a normalized tool JSON object does not recover the model's original sampled sequence or its behavior distribution.

Pin one adapter/checkpoint for an entire rollout. Record collection and consumed versions; apply a reviewed age or divergence rule before updates. A trajectory spanning two policy versions needs explicitly defined handling, not one invented "old" probability. Proximal's October 8 account fixes an adapter per rollout, captures sampled tokens near inference, and excludes overly stale groups. Its private implementation and gains are not reproduced here. Proximal infrastructure.

Question. A receipt has exact HTTP bytes, aggregate input/output counts, reward, and readable actions. Is it PPO-ready? **Solution.** No. Counts do not identify the sampled IDs, masks, or per-token behavior log-probabilities. An API's available probability field also needs a verified interpretation for the actual sampling path. Honest grouped exports remain candidate records, not trainable batches.

Hosted training has a separate availability gate. At the access date, OpenAI's RFT guide says its fine-tuning platform is winding down and is closed to new users. Do not assume an account, supported model, accepted rollout format, or permission to create a job. Dated RFT guide.

Deliverable. A batch admission checklist with exact tensor fields, version/staleness rules, group membership, zero-variance handling, and fail-closed missing-field behavior. No optimization step or trained checkpoint is produced.

Lab 7. Budget inference sessions, not only host tasks

Host concurrency counts active rollout tasks. GPU serving concurrency counts requests or sessions that occupy model compute and memory. Neither is equivalent to environments per second. A rollout can spend most of its wall time waiting for inference, holding memory or a sandbox while using little CPU.

Language-model inference has a **prefill** phase that processes input tokens and a **decode** phase that generates new tokens. Attention stores key/value tensors, called KV cache, so later tokens can reuse earlier attention state. Prefix caching reuses a shared input prefix. vLLM's documentation explicitly distinguishes this prefill saving from generating new output tokens. Prefix caching.

Solve a simplified KV budget

For a dense decoder with L layers, H_{KV} key/value heads, head dimension d, b bytes per stored scalar, and n cached tokens, approximate unsharded KV storage as

$$M_{\mathrm{KV}} \approx 2LH_{\mathrm{KV}}dbn.$$

The factor two accounts for keys and values. Suppose $L = 32$, $H_{\mathrm{KV}} = 8$, $d = 128$, and $b = 2$. Each token needs 131,072 bytes, or 128 KiB. An 8192-token sequence needs about 1 GiB. If the **remaining KV budget after weights/workspaces** is 24 GiB, at most 24 such unshared sequences fit under this simplified bound. At 16,384 tokens, that drops to twelve.

Actual layouts may use sharding, quantization, shared prefixes, paging, other attention architectures, and allocator overhead. Those require the served model's measured configuration. PagedAttention addresses KV memory management, not an unlimited-session promise. This arithmetic is not a measurement of the installed Qwen model. PagedAttention paper.

Diagnose a queue and a bottleneck

In a stable system, Little's law gives average in-flight work $C = \lambda W$, where λ is completion rate and W is average time in the measured system. Ten completed episodes per second with forty seconds average residence implies 400 live episodes. Some may be parked or evicted rather than retaining full KV; that choice changes latency and recomputation. It does not disappear from accounting.

Suppose an environment executor supports four episodes per second, while an inference service generates 400 output tokens per second and each episode needs 500 output tokens. Ignoring prefill and other overhead yields an inference ceiling of 0.8 episodes per second. Raising environment throughput to eight cannot raise end-to-end throughput above that ceiling. This is an upper bound, not a predicted benchmark.

Measure queued, admitted, prefill, decode, tool wait, grading, persistence, and cleanup durations separately. Report warm/cold state, input/output lengths, actual peak concurrency, and tail latencies. A host semaphore cannot tell a serving replica how much a long session's context will grow.

Preserve the intended workload mix

Prefix affinity can improve reuse but may repeatedly favor short, common tasks. Long-context sessions occupy KV longer and can starve. If the trainer requested half simple and half complex groups, consuming 90% simple completions changes the training distribution even when raw throughput rises. Admission needs capacity reservations, predicted growth, queue age, and declared mixing rules. MISA-T studies KV-residency-aware admission while preserving mixed workloads; its paper's measured gains are not this project's results. MISA-T.

Question. A dashboard reports 99% prefix-cache hits. Can decode be faster by 99%? **Solution.** No. A hit indicates reusable input state under a declared metric. Decode work, cache lookup/eviction, memory contention, and output length remain. Track saved prefill time and completed useful groups, not a hit percentage alone.

Deliverable. A session-memory budget with overhead reserve, admission and overload rules, phase timings, workload proportions, and cancellation/recovery tests. The local concurrency and Docker receipts do not establish a distributed GPU scheduler.

Lab 8. Price a usable result with explicit unknowns

Count all attempted work, including failures, rejected candidates, setup, grading, and cleanup. A cost per successful clean trace needs a denominator that defines useful output. A cheap unsafe episode is not a saving. Model quality, reward calibration, and data rights can dominate hardware cost.

For a declared hypothetical price sheet, write

$$C_{\text{attempt}} = \frac{I_u p_u + I_c p_c + O p_o}{10^6} + t_s p_s + C_{\text{grade}} + C_{\text{storage}} + C_{\text{other}}.$$

I_u and I_c are uncached and billed-cached input tokens, O is output tokens, and token prices are dollars per million. $t_s p_s$ prices sandbox residence in seconds. Add cache-write charges, minimum billing intervals, retries, network transfer, and provisioned idle capacity when applicable. Internal prefix reuse and provider invoice categories are different facts. The equation is not any vendor's current quote.

Solve a per-attempt example

Assume twenty calls, each with 3000 uncached input and 200 output tokens. Use invented prices of \$2 per million input and \$8 per million output. Model cost is $60{,}000 \times 2/10^6 + 4000 \times 8/10^6 = \$0.152$. Forty sandbox seconds at an invented \$0.001 per second add \$0.04. A hypothetical \$0.01 grader charge gives \$0.202 per attempt, before omitted costs.

If 20% of attempts yield a clean, rights-approved, useful trace, expected cost per such trace is $0.202/0.2 =$ \$1.01 under steady averages. Halving sandbox residence reduces that to $0.182/0.2 =$ \$0.91. Doubling useful yield to 40% instead gives $0.202/0.4 =$ \$0.505, provided the accepted traces have equal quality and no hidden selection cost. Better data yield can matter more than faster setup.

If 90% of input tokens receive a hypothetical cached price of \$0.50 per million, input cost becomes $6000 \times 2/10^6 + 54{,}000 \times 0.5/10^6 =$ \$0.039. That calculation requires actual invoice-category counts and compatible cache conditions. It cannot use a GPU prefix-hit chart as a substitute.

Solve a break-even example

Suppose building and maintaining a simulator costs an invented $12,000, and two already-calibrated approaches produce equivalent usable outcomes at $1.50 and $1.01 each. Ignoring future maintenance differences, break-even is $12{,}000/(1.50-1.01) \approx 24{,}490$ usable outcomes. If their fidelity, curation quality, failure risk, or ongoing costs differ, the comparison is invalid until those differences enter the model.

For self-hosted inference, allocated GPU-hour cost includes idle capacity and failed work. Divide by completed accepted output over the same interval. An already-owned local computer has no API invoice, but electricity, occupancy, hardware, and operator time are not mathematically zero. Do not equate zero marginal API charge with commercial unit economics.

Solve incomplete accounting

Ten finalized calls report usage for only six. Adding those six counts yields a known partial sum, not a complete cost. Without hard per-call token caps and an applicable billing rule, missing counts have no justified finite dollar upper bound. With caps, construct a conservative interval and label its assumptions. A per-episode timeout is not a money cap, particularly when provider retries or delayed cancellation can still incur charges.

Keep prices, region, cache policy, billing unit, currency, collection interval, useful-output definition, and omitted items in the receipt. Use an explicit spend cap and authorized stop rule for any hosted experiment. A trial budget authorizes neither more accounts nor a later automatic rerun.

Deliverable. A spreadsheet-ready cost equation, three sensitivity cases, equal-utility assumptions, missing-usage interval, and a stop rule. Actual matched-study usage is explicitly incomplete, so this workbook does not calculate its billed cost or a competitor advantage.

Lab 9. Review a deployment as several trust boundaries

A threat model names assets, actors, attacker capabilities, trust assumptions, controls, and residual risk. "Runs in a container" answers only part of that model. The policy, environment service, grader, artifact collector, inference server, storage, and control API have different permissions and failure modes.

Produce a permission matrix

Component	Allowed data/operations	Must not receive
Policy/agent	Public task, authorized observations, bounded tools	Hidden tests, expected final state, control-plane credentials
Environment service	Own task state; enforce scoped transitions	Cross-tenant state or unrestricted host commands
Grader	Trusted initial facts, detached state/history	Agent-controlled scoring code or network authority
Artifact collector	Read frozen output, enforce complete archive bounds	Answer keys; privileged mounts; writable agent continuation
Inference service	Authorized model context and serving weights	Grader secrets or unrelated tenants' prompts
Controller	Admit/cancel jobs; own cleanup; persist receipts	Unchecked user-selected image, mounts, or daemon flags
Operator API	Authorized tenant-scoped detail and exports	Anonymous access to private snapshots or raw traces

Harbor documents separate verification as an opt-in mode; default shared verification is not automatically isolated. Docker's security guidance treats the daemon and configuration as privileged concerns. gVisor adds a boundary against untrusted-code exploitation but documents broader attack classes and limits. Harbor verifier, Docker security, gVisor security model.

Solve a boundary exercise

An agent cannot write the hidden grader file, but can write arbitrary archive entries. It emits a symbolic link named `summary.csv`, plus 500 MiB under unrelated names. Reading only the selected filename can follow the link or miss resource exhaustion. Validate the complete detached archive before selecting output: reject links, traversal, devices, duplicate/truncated entries, unsupported formats, and excess total files/bytes. Freeze the agent first so checked files cannot change between inspection and use.

A separate grader that still mounts the agent's writable workspace can retain a race or import-path attack. A nonroot agent with the Docker socket can retain host authority. A memory cgroup can limit resource use without proving data isolation. Hashing a receipt proves byte consistency relative to a known hash, not that a privileged host produced honest facts.

Local synthetic Python worlds are trusted objects in one process; they do not sandbox arbitrary Python. The existing Docker executor's constraints and cleanup receipts are bounded evidence, not a kernel-escape guarantee or multi-tenant service. Full operator snapshots remain sensitive even when policy observations are partial and training exports are disabled.

Require evidence before deployment

Use the following gates as an engineering review, not a certification claim. NIST's control catalog provides a broader security/privacy reference. NIST SP 800-53.

1. Authenticate every control and export request; derive tenant scope from trusted identity rather than a client field.
2. Prove object-level authorization across database queries, artifacts, replay, and downloads. Test guessed IDs and mixed-tenant candidate pools.
3. Separate tenant credentials, filesystem roots, caches, logs, network policies, and retention. Review cache side channels and raw request leakage.
4. Pin reviewed images/dependencies, prevent arbitrary daemon parameters, and bound CPU, memory, PIDs, output, time, queued work, and spending.
5. Make cancellation, provider closure, persistence, and resource destruction observable. Preserve failures; release admission even after cleanup errors.
6. Exercise restart and crash recovery. Never resume an unknown live process or relaunch a paid attempt without a declared rule.
7. Complete the behavioral-fidelity study, independent grader red-team tests, expert outcome calibration, family-heldout evaluation, and rights review.
8. Enforce retention and deletion across source records, derived traces, exports, backups, and access logs. Deleting a row does not prove removal of its influence from an already-trained model.

Question. Is a loopback-only API with no authentication safe to expose through a reverse proxy? **Solution.** No. Loopback location is a deployment assumption, not user identity or authorization. Adding TLS protects transport but does not prevent one authenticated tenant from reading another's run.

Deliverable. A component permission matrix, attack tree, residual-risk list, and readiness checklist whose items each link to a test or operational proof. Mark missing evidence as blocked. A roadmap entry, successful reference rollout, schema-valid trajectory, or polished console cannot clear a deployment gate by itself.